

Contents

1	EKS Cluster Security Validation Runbook	1
1.1	Overview	1
1.2	Prerequisites	2
1.3	1 Pod Security Admission (PSA) Validation	2
1.3.1	What It Does	2
1.3.2	How to Check	2
1.4	2 Pod Security Context Validation	2
1.4.1	What It Does	2
1.4.2	How to Check	3
1.5	3 Resource Limits Validation	3
1.5.1	What It Does	3
1.5.2	How to Check	3
1.6	4 KMS Encryption Validation	4
1.6.1	What It Does	4
1.6.2	How to Check	4
1.7	5 Network Access Validation	5
1.7.1	What It Does	5
1.7.2	How to Check	5
1.8	6 Control Plane Logging Validation	5
1.8.1	What It Does	5
1.8.2	How to Check	5
1.9	7 Network Policies Validation	6
1.9.1	What It Does	6
1.9.2	How to Check	6
1.10	8 Application Health Validation	7
1.10.1	What It Does	7
1.10.2	How to Check	7
1.11	9 IAM Roles Validation	7
1.11.1	What It Does	7
1.11.2	How to Check	7
1.12	Node Security Validation	8
1.12.1	What It Does	8
1.12.2	How to Check	8
1.13	Security Scorecard	9
1.14	Quick Validation Script	9
1.15	Troubleshooting	11
1.15.1	Issue: PSA prevents pod from starting	11
1.15.2	Issue: Network policy blocks all traffic	11
1.15.3	Issue: KMS encryption not showing	12
1.15.4	Issue: Control plane logs not appearing	12
1.16	Additional Resources	12
1.17	Validation Checklist	12

1 EKS Cluster Security Validation Runbook

1.1 Overview

This runbook provides step-by-step instructions to validate the security configuration of your EKS cluster. Each check includes the command to run, what to look for, and how to interpret the results.

1.2 Prerequisites

Before starting, ensure you have: - [] AWS CLI configured with proper credentials - [] kubectl installed and configured - [] Access to the EKS cluster (run: `kubectl cluster-info`) - [] jq installed (optional, for better JSON parsing): `brew install jq`

Quick check:

```
# Verify connectivity
kubectl cluster-info
```

```
# Expected: Should show cluster endpoint and services
```

1.3 1 Pod Security Admission (PSA) Validation

1.3.1 What It Does

Ensures that pods run with security restrictions (no root access, no privileged mode, etc.)

1.3.2 How to Check

Step 1: Verify namespace labels

```
kubectl get namespace production -o yaml | grep -A 5 "labels:"
```

What to look for: - `pod-security.kubernetes.io/enforce: restricted` - `pod-security.kubernetes.io/audit: restricted` - `pod-security.kubernetes.io/warn: restricted`

Step 2: Test PSA enforcement

```
# Try to create a privileged pod (should fail)
```

```
kubectl apply -f - <<EOF
```

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: privileged-test
```

```
  namespace: production
```

```
spec:
```

```
  containers:
```

```
  - name: test
```

```
    image: nginx
```

```
    securityContext:
```

```
      privileged: true
```

```
EOF
```

Expected result:

Error: pods "privileged-test" is forbidden: violates PodSecurity "restricted:latest"

PASS: Error means PSA is working correctly!

1.4 2 Pod Security Context Validation

1.4.1 What It Does

Verifies that pods are running as non-root users with proper security settings

1.4.2 How to Check

Step 1: Check all running pods

```
kubectl get pods -n production -l app=secure-nginx -o wide
```

What to look for: - STATUS should be Running - READY should show 1/1 or 2/2 (all containers ready)

Step 2: Verify pod security context

```
kubectl get pod -n production -l app=secure-nginx -o json | \
jq -r '.items[0].spec.securityContext'
```

What to look for:

```
{
  "fsGroup": 1000,
  "runAsNonRoot": true,
  "runAsUser": 1000,
  "seccompProfile": {
    "type": "RuntimeDefault"
  }
}
```

- runAsNonRoot: true - Pod cannot run as root
- runAsUser: 1000 - Specific non-root user ID
- seccompProfile - System call filtering enabled

Step 3: Verify container security context

```
kubectl get pod -n production -l app=secure-nginx -o json | \
jq -r '.items[0].spec.containers[0].securityContext'
```

What to look for:

```
{
  "allowPrivilegeEscalation": false,
  "capabilities": {
    "drop": ["ALL"]
  },
  "readOnlyRootFilesystem": true,
  "runAsNonRoot": true
}
```

- allowPrivilegeEscalation: false - Cannot gain more privileges
- capabilities.drop: ["ALL"] - All Linux capabilities removed
- readOnlyRootFilesystem: true - Cannot write to filesystem

1.5 3 Resource Limits Validation

1.5.1 What It Does

Ensures pods have CPU and memory limits to prevent resource exhaustion

1.5.2 How to Check

```
kubectl get pod -n production -l app=secure-nginx -o json | \
jq -r '.items[0].spec.containers[0].resources'
```

What to look for:

```
{
  "limits": {
    "cpu": "500m",
    "memory": "512Mi"
  },
  "requests": {
    "cpu": "250m",
    "memory": "256Mi"
  }
}
```

- Both `limits` and `requests` are defined
- Limits prevent resource hogging
- Requests ensure minimum guaranteed resources

Quick visual check:

```
kubectl top pods -n production
```

Shows actual CPU and memory usage vs limits.

1.6 4 KMS Encryption Validation

1.6.1 What It Does

Verifies that Kubernetes secrets are encrypted at rest using AWS KMS

1.6.2 How to Check

Step 1: Check cluster encryption configuration

```
aws eks describe-cluster --name prod-secure-eks-cluster \
  --query 'cluster.encryptionConfig' --output table
```

What to look for:

```
-----+
|          EncryptionConfig          |
+-----+
||          Resources                ||
|+-----+
|| Provider | Key ARN                ||
|+-----+
|| KMS      | arn:aws:kms:...        ||
|+-----+

```

- `resources: ["secrets"]` present
- `provider.keyArn` points to a KMS key

Step 2: Verify KMS key details

Get the key ARN from above, then:

```
aws kms describe-key --key-id <KEY_ARN> \
  --query 'KeyMetadata.{KeyState:KeyState,Enabled:Enabled,Origin:Origin}' \
  --output table
```

What to look for: - KeyState: Enabled - Enabled: true

Step 3: Check key rotation

```
aws kms get-key-rotation-status --key-id <KEY_ARN>
```

What to look for:

```
{
  "KeyRotationEnabled": true
}
```

- Automatic key rotation is enabled (best practice)
-

1.7 5 Network Access Validation

1.7.1 What It Does

Ensures the EKS cluster API endpoint has proper access controls

1.7.2 How to Check

Step 1: Check endpoint configuration

```
aws eks describe-cluster --name prod-secure-eks-cluster \
  --query 'cluster.resourcesVpcConfig.{PrivateAccess:endpointPrivateAccess,PublicAccess:endpointPublicAccess}' \
  --output table
```

What to look for:

Production (most secure): - PrivateAccess: true - PublicAccess: false - Cluster only accessible from within VPC

Development (balanced): - PrivateAccess: true - PublicAccess: true (acceptable for dev) - PublicCIDRs: ["<your-ip>/32"] (restricted to your IP)

Security Risk: - PublicAccess: true + PublicCIDRs: ["0.0.0.0/0"] (open to internet)

1.8 6 Control Plane Logging Validation

1.8.1 What It Does

Verifies that EKS control plane activities are logged to CloudWatch

1.8.2 How to Check

Step 1: Check enabled log types

```
aws eks describe-cluster --name prod-secure-eks-cluster \
  --query 'cluster.logging.clusterLogging[0].types' --output table
```

What to look for:

```
-----
|      Types      |
+-----+
|  api            |
|  audit          |
|  authenticator  |
+-----+
```

```
| controllerManager |
| scheduler          |
+-----+
```

- All 5 log types should be enabled

Step 2: Verify CloudWatch log group

```
aws logs describe-log-groups \
  --log-group-name-prefix /aws/eks/prod-secure-eks-cluster \
  --query 'logGroups[0].{Name:logGroupName,Retention:retentionInDays,Size:storedBytes}' \
  --output table
```

What to look for: - Log group exists - retentionInDays is set (e.g., 90 days) - storedBytes > 0 (logs are being written)

Step 3: Check recent logs

```
aws logs tail /aws/eks/prod-secure-eks-cluster/cluster --follow --since 5m
```

- Should see recent log entries

1.9 7 Network Policies Validation

1.9.1 What It Does

Tests that network policies correctly allow/block traffic between pods

1.9.2 How to Check

Step 1: Verify network policies exist

```
kubectl get networkpolicies -n production
```

What to look for:

NAME	POD-SELECTOR	AGE
allow-test-client	app=secure-nginx	10m
default-deny-ingress	<none>	10m

- At least one network policy exists

Step 2: Deploy test clients

Allowed client (has matching label)

```
kubectl run test-client -n production --image=busybox --labels="role=test" -- sleep 3600
```

Blocked client (no matching label)

```
kubectl run blocked-client -n production --image=busybox -- sleep 3600
```

Step 3: Test connectivity from allowed client

```
kubectl exec -n production test-client -- wget -q -O- --timeout=5 http://secure-nginx-service
```

- **PASS:** Returns HTML content
- **FAIL:** Timeout or connection refused

Step 4: Test connectivity from blocked client

```
kubectl exec -n production blocked-client -- wget -q -O- --timeout=5 http://secure-nginx-service
```

- **PASS:** Timeout or connection refused
- **FAIL:** Returns HTML content

Note: Network policies require a CNI plugin like Calico. If both clients succeed, your CNI may not support network policies.

Check if Calico is installed:

```
kubect1 get pods -n kube-system | grep calico
```

1.10 8 Application Health Validation

1.10.1 What It Does

Verifies the deployed application is running correctly with security features

1.10.2 How to Check

Step 1: Check pod status

```
kubect1 get pods -n production -l app=secure-nginx
```

- All pods should be Running and Ready

Step 2: Check deployment status

```
kubect1 get deployment -n production secure-nginx
```

- READY should match DESIRED (e.g., 2/2)

Step 3: Verify the pods are NOT running as root

```
kubect1 exec -n production -l app=secure-nginx -- whoami
```

- Should return nginx or www-data or user ID 1000
- Should NOT return root

Step 4: Test application endpoint

```
kubect1 port-forward -n production svc/secure-nginx-service 8080:80 &  
curl http://localhost:8080
```

- Should return nginx welcome page HTML
- Response time should be < 100ms

Step 5: Check pod events for warnings

```
kubect1 describe pod -n production -l app=secure-nginx | grep -A 10 Events:
```

- No Warning or Error events
 - Only Normal events (Scheduled, Pulled, Created, Started)
-

1.11 9 IAM Roles Validation

1.11.1 What It Does

Verifies IAM roles have appropriate permissions and lifecycle protection

1.11.2 How to Check

Step 1: Check cluster IAM role

```
aws iam get-role --role-name eks-cluster-role \
  --query 'Role.{RoleName:RoleName,Created:CreateDate,Policies:AttachedPolicies}' \
  --output table
```

What to look for: - Role exists - Has AmazonEKSClusterPolicy attached

Step 2: Check node group IAM role

```
aws iam get-role --role-name eks-node-role \
  --query 'Role.{RoleName:RoleName,Created:CreateDate}' \
  --output table
```

What to look for: - Role exists - Has the following policies attached: - AmazonEKSWorkerNodePolicy - AmazonEKS_CNI_Policy - AmazonEC2ContainerRegistryReadOnly

Step 3: Verify Terraform lifecycle protection

```
grep -A 3 "lifecycle" /Users/sunils/eks-cluster-project/PHASE2/phase1-basic.tf | grep prevent_destroy
```

What to look for: - prevent_destroy = true for both IAM roles - This prevents accidental deletion via Terraform

1.12 Node Security Validation

1.12.1 What It Does

Checks the security configuration of worker nodes

1.12.2 How to Check

Step 1: List nodes

```
kubectl get nodes -o wide
```

What to look for: - All nodes show Ready status - Kubernetes version matches cluster version

Step 2: Check node security groups

```
aws eks describe-nodegroup \
  --cluster-name prod-secure-eks-cluster \
  --nodegroup-name <nodegroup-name> \
  --query 'nodegroup.{AMITYPE:amiType,InstanceTypes:instanceTypes,ScalingConfig:scalingConfig}' \
  --output table
```

What to look for: - amiType is AL2_x86_64 or AL2_ARM_64 (Amazon Linux 2) - instanceTypes are appropriate for workload

Step 3: Verify nodes are not publicly accessible

```
aws ec2 describe-instances \
  --filters "Name=tag:eks:cluster-name,Values=prod-secure-eks-cluster" \
  --query 'Reservations[*].Instances[*].{InstanceId:InstanceId,PrivateIP:PrivateIpAddress,PublicIP:PublicIpAddress}' \
  --output table
```

What to look for: - PrivateIP exists - PublicIP should be empty/null (nodes in private subnets) - If PublicIP exists, nodes are in public subnets (not recommended)

1.13 Security Scorecard

After completing all checks, tally your results:

Check	Status	Weight
PSA enforced (restricted profile)		15%
Pod security contexts configured		10%
Container security contexts configured		10%
Resource limits defined		5%
KMS encryption enabled		15%
KMS key rotation enabled		5%
Private endpoint access enabled		10%
Public endpoint restricted (or disabled)		10%
Control plane logging (all 5 types)		10%
Network policies configured		5%
IAM roles with prevent_destroy		5%

Scoring: - **90-100%:** Excellent - Production ready - **75-89%:** Good - Minor improvements needed - **60-74%:** Fair - Several security gaps - **< 60%:** Poor - Major security issues

1.14 Quick Validation Script

Run all checks automatically:

```
#!/bin/bash
# Save as: validate-cluster-security.sh

CLUSTER_NAME="prod-secure-eks-cluster"
NAMESPACE="production"
SCORE=0
TOTAL=11

echo " EKS Cluster Security Validation"
echo "===== "
echo ""

# Check 1: PSA
if kubectl get ns $NAMESPACE -o yaml | grep -q "pod-security.kubernetes.io/enforce: restricted"; then
    echo " PSA enforced"
    ((SCORE++))
else
    echo " PSA not enforced"
fi

# Check 2: Pod security context
if kubectl get pod -n $NAMESPACE -l app=secure-nginx -o json | jq -e '.items[0].spec.securityContext.runAsNonRoot'; then
    echo " Pod runs as non-root"
    ((SCORE++))
else
    echo " Pod security context missing"
fi
```

```

# Check 3: Container security context
if kubectl get pod -n $NAMESPACE -l app=secure-nginx -o json | jq -e '.items[0].spec.containers[0].securityContext'
    echo " Container privilege escalation disabled"
    ((SCORE++))
else
    echo " Container security context missing"
fi

# Check 4: Resource limits
if kubectl get pod -n $NAMESPACE -l app=secure-nginx -o json | jq -e '.items[0].spec.containers[0].resources'
    echo " Resource limits defined"
    ((SCORE++))
else
    echo " Resource limits missing"
fi

# Check 5: KMS encryption
if aws eks describe-cluster --name $CLUSTER_NAME --query 'cluster.encryptedConfig' | grep -q "secrets"
    echo " KMS encryption enabled"
    ((SCORE++))
else
    echo " KMS encryption not enabled"
fi

# Check 6: Private endpoint
if aws eks describe-cluster --name $CLUSTER_NAME --query 'cluster.resourcesVpcConfig.endpointPrivateAccessEnabled'
    echo " Private endpoint enabled"
    ((SCORE++))
else
    echo " Private endpoint disabled"
fi

# Check 7: Public endpoint restriction
PUBLIC_CIDRS=$(aws eks describe-cluster --name $CLUSTER_NAME --query 'cluster.resourcesVpcConfig.publicAccessCidrs')
if [[ "$PUBLIC_CIDRS" != "0.0.0.0/0" ]]; then
    echo " Public endpoint restricted"
    ((SCORE++))
else
    echo " Public endpoint open to internet"
fi

# Check 8: Control plane logging
LOG_TYPES=$(aws eks describe-cluster --name $CLUSTER_NAME --query 'cluster.logging.clusterLogging[0].types')
if [[ "$LOG_TYPES" -eq 5 ]]; then
    echo " All control plane logs enabled"
    ((SCORE++))
else
    echo " Control plane logging incomplete ($LOG_TYPES/5)"
fi

# Check 9: Network policies
if kubectl get networkpolicies -n $NAMESPACE &>/dev/null && [[ $(kubectl get networkpolicies -n $NAMESPACE) ]]
    echo " Network policies configured"
    ((SCORE++))

```

```

else
    echo " No network policies found"
fi

# Check 10: Pods running
if kubectl get pods -n $NAMESPACE -l app=secure-nginx &>/dev/null && [[ $(kubectl get pods -n $NAMESPACE)
    echo " Application pods running"
    ((SCORE++))
else
    echo " No running pods found"
fi

# Check 11: CloudWatch logs
if aws logs describe-log-groups --log-group-name-prefix /aws/eks/$CLUSTER_NAME | grep -q "logGroupName"
    echo " CloudWatch log group exists"
    ((SCORE++))
else
    echo " CloudWatch log group missing"
fi

echo ""
echo "=====
PERCENTAGE=$((SCORE * 100 / TOTAL))
echo "Security Score: $SCORE/$TOTAL ($PERCENTAGE%)"

if [ $PERCENTAGE -ge 90 ]; then
    echo " Status: Excellent"
elif [ $PERCENTAGE -ge 75 ]; then
    echo " Status: Good"
elif [ $PERCENTAGE -ge 60 ]; then
    echo " Status: Fair"
else
    echo " Status: Poor"
fi
echo "=====

Make it executable and run:

chmod +x validate-cluster-security.sh
./validate-cluster-security.sh

```

1.15 Troubleshooting

1.15.1 Issue: PSA prevents pod from starting

Symptom: Pod shows `CreateContainerConfigError` or fails validation

Solution: 1. Check pod spec against restricted profile requirements 2. Ensure `runAsNonRoot: true` is set 3. Use unprivileged container images (e.g., `nginxinc/nginx-unprivileged`)

1.15.2 Issue: Network policy blocks all traffic

Symptom: Cannot connect to application even from allowed clients

Solution: 1. Verify CNI plugin supports network policies: `kubectl get pods -n kube-system | grep calico` 2. Check policy selectors match pod labels 3. Temporarily disable network policy to isolate issue

1.15.3 Issue: KMS encryption not showing

Symptom: `describe-cluster` shows no encryptionConfig

Solution: 1. Check Terraform apply completed successfully 2. Verify KMS key exists and is enabled 3. Note: Encryption only applies to new secrets created after enabling

1.15.4 Issue: Control plane logs not appearing

Symptom: CloudWatch log group is empty

Solution: 1. Wait 5-10 minutes for logs to appear 2. Check log group retention is not set too short 3. Verify cluster has activity (deploy pods, run kubectl commands)

1.16 Additional Resources

- [EKS Best Practices Guide](#)
 - [Pod Security Standards](#)
 - [Network Policies Documentation](#)
 - [KMS Encryption for EKS](#)
-

1.17 Validation Checklist

Print this checklist and check off items as you validate:

- ☐ Prerequisites verified (kubectl, aws cli, cluster access)
- ☐ PSA enforcement tested and working
- ☐ Pod security contexts validated
- ☐ Container security contexts validated

- ☐ Resource limits verified
- ☐ KMS encryption enabled and key rotation active
- ☐ Network access properly configured
- ☐ Control plane logging enabled (all 5 types)
- ☐ CloudWatch logs visible
- ☐ Network policies deployed and tested
- ☐ Application running and healthy
- ☐ IAM roles have lifecycle protection
- ☐ Nodes in private subnets (no public IPs)
- ☐ Security scorecard completed
- ☐ Quick validation script executed

Date validated: _____

Validated by: _____

Overall score: _____

Last updated: November 9, 2025